

Software Engineering II

CQRS

Mathis Neunzig
ABAP Developer at SAP
dhbw@mathis-neunzig.de



Architekturmuster

Was wir heute in der VL machen
ist grün markiert!

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

Command Query Responsibility Segregation

- CQRS bedeutet: Commands und Queries werden getrennt behandelt
- Commands ändern den Zustand, Queries nur lesen (keine Seiteneffekte)
- Ziel: klare Trennung von Lese- und Schreiblogik, weniger Komplexität pro Modul
- Oft in Systemen mit hoher Last oder vielen gleichzeitigen Zugriffen genutzt
- Hilft bei Performance-Tuning, da Lese- und Schreibseiten unabhängig skalieren
- Beispiele: Bestellung aufgeben (Command), Bestellstatus abrufen (Query)

Vorteile

- Leseseite kann optimiert werden (z. B. eigene Datenbank oder Cache)
- Schreibseite kann gezielt mit Validierung und Business-Logik versehen werden
- Weniger Konflikte und einfachere Datenmodelle pro Seite
- Besser geeignet für Event Sourcing oder Event-Architekturen
- Flexibler: unterschiedliche Skalierung für Lesen (z. B. viele Kundenanfragen) und Schreiben (z. B. nur bei Bestellungen)
- Beispiel: Bestellstatus wird häufig abgerufen, aber nur selten geändert

CQRS mit RabbitMQ

- Producer sendet Command-Events an Exchange, z. B. „create.order“
- Consumer für Commands: Bestellservice verarbeitet „create.order“ und speichert Daten
- Separater Query-Service abonniert Events wie „order.created“ und aktualisiert eigene Lesemodelle
- Beispiel: Lager-Service erhält „order.created“ und passt Bestand an, Query-Service aktualisiert „orders.readmodel“
- RabbitMQ sorgt für zuverlässige Verarbeitung und Entkopplung der Services

Beispiel

- Kunde gibt Bestellung auf → Producer sendet Command „create.order“ an RabbitMQ
- Command-Consumer: Bestellservice erstellt Order und veröffentlicht Event „order.created“
- Query-Consumer: Query-Service abonniert „order.created“ und speichert Daten für schnelle Abfragen
- Leseseite: Kunde ruft Bestellübersicht ab (Query-Service liest Readmodel)
- Vorteil: Readmodel kann z. B. in Elasticsearch oder Redis liegen – optimiert für schnelle Abfragen
- RabbitMQ verbindet Command- und Query-Services asynchron, stabil und skalierbar

CQRS & EDA

- EDA liefert Events (z. B. „order.created“), die als Trigger für CQRS-Commands und Queries dienen
- Commands werden durch Events ausgelöst oder als Events versendet, Queries konsumieren aktualisierte Datenmodelle
- Beispiel: „create.order“ Event löst Command-Handler aus (Write), Query-Consumer hört auf „order.created“ (Read)
- RabbitMQ oder Kafka vermittelt Events zwischen Commands und Query-Services
- CQRS-Readmodel wird oft durch Event-Streams (EDA) automatisch aktualisiert
- Ergebnis: hochskalierbare Systeme mit klarer Rollenverteilung – Events verbinden Write- und Readmodel in CQRS

Aufgabe: EDA mit CQRS

- Entwickel ein kleines Softwaresystem mit EDA und CQRS
- Mögliche Szenarien:
 - Der STOP-Knopf wird im Bus gedrückt (Sound abspielen, “STOP” aufleuchten, ...)
 - Sensor erkennt, dass man zuhause ist und schaltet Licht und Heizung an, etc.
 - Ein Produkt im Supermarkt wird gekauft (Neue Bestellung aufgeben, ...)
- Clean-Code Regeln beachten!
- Abgabe über GitHub
- Abgabe bis zum Start der nächsten Vorlesung
- Mögliche Punkte: 5+5=10
- Zeit: tbd

Danke!

Mathis Neunzig
dhbw@mathis-neunzig.de

